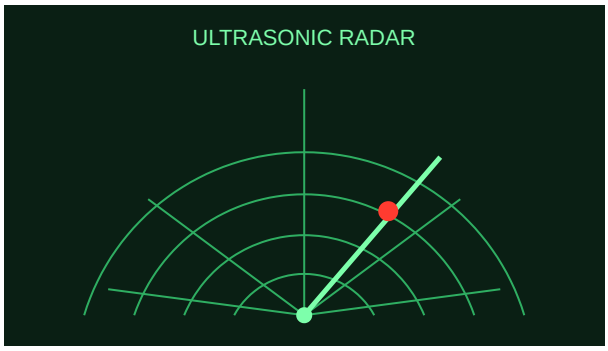
SEMESTER PROJECT REPORT



Arduino Ultrasonic Radar System

Design, implementation and testing of a low-cost scanning ultrasonic radar using an Arduino UNO, an HC-SR04 sensor, an SG90 servo motor and a Processing radar display

Prepared by: Ali Mursal Shah

Programme: BS Electronics

Institution: COMSATS University Islamabad

This report documents the complete project: the hardware design and wiring, the distance calculation, the full Arduino and Processing code, the setup and testing procedure, and an evaluation of the system's limitations and future scope.

Contents

Abstract	2
1. Introduction and Background	3
2. Problem Statement	3
3. Objectives	3
4. Scope of the Project	4
5. System Overview	4
6. Components Required	5
7. Hardware Design and Wiring	5
8. Working Principle	6
9. Theory and Calculations	6
10. Arduino Firmware	7
11. Processing Radar Display	8
12. Setup and Operating Procedure	10
13. Testing and Evaluation	10
14. Troubleshooting	11
15. Project Timeline	11
16. Estimated Budget	12
17. Limitations and Risks	12
18. Applications	13
19. Future Scope	13
20. Conclusion	13
21. References	13

Abstract

This report describes the design and implementation of a low-cost ultrasonic radar system built around an Arduino UNO. An HC-SR04 ultrasonic sensor is mounted on an SG90 servo motor that sweeps through an arc of 15 degrees to 165 degrees. At each angle, the Arduino measures the distance to the nearest object and sends the angle and distance pair to a computer over USB serial communication at 9600 baud. A Processing application then draws a radar-style display showing the sweep line, range rings up to 200 cm, and a red marker for each detected object.

The project combines embedded programming, sensor interfacing, servo control, serial communication and data visualisation in one practical, affordable prototype. The report presents the hardware design and wiring, the distance calculation, the full program listings, a step-by-step setup and testing procedure, and a discussion of accuracy, limitations and future improvements.

Keywords: Arduino UNO; HC-SR04; ultrasonic ranging; servo motor; serial communication; Processing.

1. Introduction and Background

Radar systems detect objects by emitting a signal and measuring the time taken for its reflection to return. Real radar uses radio waves, but the same principle can be demonstrated clearly and safely with ultrasonic sound. The HC-SR04 sensor emits a 40 kHz ultrasonic pulse and produces an echo pulse whose width is proportional to the distance of the nearest reflecting surface. By rotating the sensor with a servo motor, a single fixed-direction sensor can scan a whole sector and build up a picture of its surroundings.

Systems of this kind are widely used in education and in simple robotics, for example for obstacle detection and area scanning on small robots. They are also an effective teaching platform, because one small project exercises digital input and output, pulse timing, motor control, serial communication and real-time graphics together.

2. Problem Statement

A single ultrasonic sensor measures distance along one fixed direction only. It gives no information about where in the surrounding area objects are located, and raw numbers on a serial monitor are hard to interpret. There is therefore a need for a simple, low-cost system that scans an area, measures distances at known angles, and presents the results visually in real time.

3. Objectives

1. Interface an HC-SR04 ultrasonic sensor and an SG90 servo motor with an Arduino UNO.
2. Sweep the sensor between 15 degrees and 165 degrees and measure the distance at each angle step.
3. Transmit angle and distance data to a computer as comma-separated serial lines at 9600 baud.
4. Develop a Processing sketch that displays the sweep line, range rings up to 200 cm, and detected objects as a radar screen.
5. Test and evaluate measurement accuracy against known distances, and document the limitations of the system.

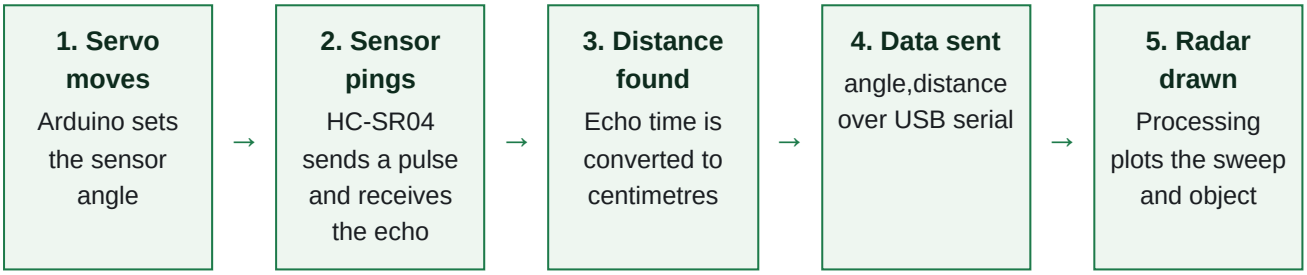
4. Scope of the Project

In scope: a breadboard prototype; a detection range displayed up to 200 cm; a scanning arc of 15 degrees to 165 degrees; a real-time display on a computer; and accuracy testing with flat, hard targets.

Out of scope: wireless data transfer, 360-degree scanning, object classification, and use as a certified safety or navigation device. The system is a learning prototype.

5. System Overview

The system has three parts. The **sensing and actuation unit** is the HC-SR04 sensor mounted on the servo motor. The **control unit** is the Arduino UNO, which positions the servo, triggers the sensor, calculates the distance and sends the data over serial. The **display unit** is a computer running Processing, which reads the serial data and draws the radar screen.



Data therefore flows in one direction in a repeating cycle: the Arduino moves the servo to an angle, the HC-SR04 sends a pulse and receives its echo, the Arduino converts the echo time into a distance, the angle and distance are sent over USB serial, and Processing draws the sweep line and the detected object. The cycle repeats as the servo sweeps up from 15 degrees to 165 degrees and back down again.

Key system parameters

Parameter	Value
Scanning arc	15 degrees to 165 degrees (150-degree sector)
Angle step	2 degrees
Display range	Up to 200 cm, with range rings every 50 cm
Serial format	angle,distance per line, for example 75,42
Serial baud rate	9600 baud (both programs must match)
No-echo reading	400 cm is reported when no echo is received within 30 ms

6. Components Required

Component	Qty.	Purpose
Arduino UNO or compatible	1	Controller and serial communication
HC-SR04 ultrasonic sensor	1	Distance measurement
SG90 (or similar) servo motor	1	Rotates the sensor
Breadboard and jumper wires	1 set	Temporary connections
USB cable	1	Code upload and serial data
Computer with Arduino IDE and Processing IDE	1	Programming and radar display
External regulated 5V supply (optional)	1	Stable servo power if the board resets or the servo jitters

7. Hardware Design and Wiring

All wiring is done with the USB cable disconnected. The HC-SR04 is mounted on the servo horn so that the servo rotation sweeps the sensor's field of view across the scanning sector.

Module / wire	Arduino connection	Notes
HC-SR04 VCC	5V	Sensor power
HC-SR04 GND	GND	Common ground
HC-SR04 TRIG	D10	Trigger output
HC-SR04 ECHO	D11	Echo input
Servo signal (orange / yellow)	D12	Control signal
Servo power (red)	5V	Or an external regulated 5V supply
Servo ground (brown / black)	GND	Must be common with the Arduino ground
Arduino USB	Computer USB	Upload and serial data

> **Power caution:** a servo can draw more current than the Arduino 5V pin can supply reliably. If the board resets or the servo jitters, power the servo from a suitable regulated 5V supply and connect that supply's ground to the Arduino ground. Do not wire a separate supply into the Arduino 5V rail without understanding the board's power arrangement.

8. Working Principle

1. The Arduino moves the servo to a chosen angle and waits briefly for it to settle.
2. The HC-SR04 is triggered and sends an ultrasonic pulse, then waits for its echo.
3. The Arduino measures the echo duration with *pulseIn()* and calculates the distance.
4. The Arduino sends the angle and distance to the computer over USB serial communication.
5. Processing reads the serial line and draws the sweep, the range rings, and a red marker for a detected object.

9. Theory and Calculations

9.1 Distance measurement

The ultrasonic pulse travels from the sensor to the object and back again, so the measured travel time corresponds to twice the distance. Using the speed of sound in air, approximately 0.0343 cm per microsecond:

$$\text{Distance (cm)} = \text{echo duration (µs)} \times 0.0343 \div 2$$

Worked example: for an echo duration of 2,000 µs, the distance is 2,000 × 0.0343 ÷ 2 = **34.3 cm**.

The firmware allows a maximum echo wait of 30 ms (30,000 µs). If no echo arrives in that time, *pulseIn()* returns zero and the sketch reports 400 cm, which lies outside the 200 cm display range, so no false object is drawn.

9.2 Servo angle

The sketch sweeps between 15 degrees and 165 degrees in 2-degree steps, first upwards and then downwards. At each step the servo is given 25 ms to move and settle before the reading is taken, and a further 20 ms pause follows each reading. The measured angle gives the direction of the distance reading, which is what turns a single distance value into a position on the radar screen.

9.3 Serial communication

Both programs use 9600 baud. Each line from the Arduino contains two values separated by a comma, such as **75,42**, meaning 75 degrees and 42 cm. Processing buffers incoming data until a newline arrives, then splits the line at the comma to recover the two values.

9.4 Display scaling

Processing maps the 0 to 200 cm range onto a radius of 42 percent of the window width. Range rings are drawn every 50 cm and angle lines every 30 degrees, so a marker's position can be read directly against the scale. Sensor accuracy in practice depends on the target surface, its angle, the temperature, and the surroundings.

10. Arduino Firmware

The sketch uses the Servo library to sweep from 15 degrees to 165 degrees and back in 2-degree steps. At each step it sends a 10 µs trigger pulse, measures the echo using *pulseIn()* with a 30 ms timeout, converts the result to centimetres, and prints *angle,distance* followed by a newline. The file is saved as **radar.ino**.

```
// Arduino Ultrasonic Radar System
// Servo on D12 sweeps HC-SR04 (TRIG D10, ECHO D11) from 15 to 165 degrees.
// Sends angle,distance over Serial at 9600 baud for the Processing display.

#include <Servo.h>

const int trigPin = 10;
const int echoPin = 11;
const int servoPin = 12;

Servo radarServo;

int readDistanceCm() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(3);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) return 400; // no echo received
  return (int)(duration * 0.0343 / 2.0);
}

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  radarServo.attach(servoPin);
  Serial.begin(9600);
}

void loop() {
  for (int angle = 15; angle <= 165; angle += 2) {
    radarServo.write(angle);
    delay(25);
    Serial.print(angle);
    Serial.print(',');
    Serial.println(readDistanceCm());
    delay(20);
  }
  for (int angle = 165; angle >= 15; angle -= 2) {
    radarServo.write(angle);
    delay(25);
    Serial.print(angle);
    Serial.print(',');
    Serial.println(readDistanceCm());
    delay(20);
  }
}
```

Verify this sketch first in the Serial Monitor at 9600 baud (for example 75,42).

11. Processing Radar Display

The Processing sketch opens the serial port, reads each line, splits it at the comma, and draws a semicircular radar with range rings every 50 cm, angle lines every 30 degrees, a sweep line at the current angle, and a red marker for any object within 200 cm. The file is saved as **radar_display.pde**.

> **Important:** close the Arduino Serial Monitor before running this sketch, because only one program can use the serial port at a time. Check the console output of *Serial.list()* and change the index [0] in the code if the Arduino is listed at a different position.

```
// Processing Radar Display - Arduino Ultrasonic Radar System
// Reads angle,distance from the Arduino over Serial at 9600 baud
// and draws a radar-style display: sweep line, range rings to 200 cm,
// angle lines every 30 degrees, and a red marker for detected objects.

import processing.serial.*;

Serial myPort;
String incoming = "";
int angle = 90, distance = 0;
int maxDistance = 200;

void setup() {
  size(1000, 600);
  smooth();
  println(Serial.list()); // Check console for Arduino port
  // Change [0] if the Arduino is listed at another index.
  myPort = new Serial(this, Serial.list()[0], 9600);
  myPort.bufferUntil('\n');
}

void draw() {
  background(0);
  translate(width/2, height*0.88);
  stroke(0, 255, 70);
  noFill();
  for (int d = 50; d <= maxDistance; d += 50) {
    float r = map(d, 0, maxDistance, 0, width*0.42);
    arc(0, 0, 2*r, 2*r, PI, TWO_PI);
  }
  for (int a = 0; a <= 180; a += 30) {
    float x = cos(radians(a))*width*0.42;
    float y = -sin(radians(a))*width*0.42;
    line(0, 0, x, y);
  }
```

Listing continued on the next page.

11. Processing Radar Display (continued)

```
  float rMax = width*0.42;
  stroke(120, 255, 150);
  line(0, 0, cos(radians(angle))*rMax, -sin(radians(angle))*rMax);
  if (distance > 0 && distance <= maxDistance) {
    float r = map(distance, 0, maxDistance, 0, rMax);
    noStroke();
    fill(255, 40, 40);
    ellipse(cos(radians(angle))*r, -sin(radians(angle))*r, 10, 10);
  }
  fill(0, 255, 70);
  textAlign(CENTER);
  textSize(16);
  text("ULTRASONIC RADAR", 0, -height*0.78);
  text("Angle: " + angle + " deg Distance: " + distance + " cm",
       0, -height*0.70);
}

void serialEvent(Serial p) {
  String line = p.readStringUntil('\n');
  if (line != null) {
    String[] values = split(trim(line), ',');
    if (values.length == 2) {
      angle = int(values[0]);
      distance = int(values[1]);
    }
  }
}
```

How the display code works

- **setup()** creates a 1000 by 600 window, prints the available serial ports to the console, opens the selected port at 9600 baud, and buffers data until a newline arrives.
- **draw()** runs continuously. It moves the drawing origin to the bottom centre of the window, draws the four range rings and the angle lines in green, then draws the sweep line at the current angle.
- **Object marker:** if the distance is greater than zero and no more than 200 cm, the distance is mapped onto the display radius and a red circle is drawn at that angle and range.
- **serialEvent()** is called automatically whenever a complete line arrives. It trims the line, splits it at the comma, and stores the angle and distance for the next frame.

12. Setup and Operating Procedure

1. Install the Arduino IDE and the Processing IDE on the computer.
2. Mount the HC-SR04 on the servo horn, then wire the sensor and servo according to the table in Section 7, with the USB cable disconnected.
3. Test the servo and the sensor individually before combining them.
4. Connect the Arduino by USB, select the board (Arduino UNO) and the correct COM port in the Arduino IDE, and upload radar.ino.
5. Open the Serial Monitor at 9600 baud and confirm that lines such as 75,42 are being printed as the servo sweeps.
6. Close the Serial Monitor. Open radar_display.pde in Processing, check the *Serial.list()* output in the console, change the port index if necessary, and run the sketch.
7. Place a broad, flat, hard object in front of the sensor and observe the sweep line and the red marker on the radar display.

13. Testing and Evaluation

Accuracy is evaluated by placing a broad, flat, hard target at known distances of 10, 20, 50, 100 and 150 cm and recording the value reported by the system at each distance. The error is the measured value minus the actual value, and the percentage error is the error divided by the actual value, multiplied by 100. The table below is the recording sheet for these measurements; it is completed from the prototype's own readings during the practical session.

Actual distance (cm)	Measured distance (cm)	Error (cm)	Error (%)	Notes
10				
20				
50				
100				
150				

The evaluation also covers:

- Testing at different angles across the sweep arc, to compare detection at the centre and at the edges of the sector.
- Comparing results for different surfaces - hard and flat, angled, and soft - to show how the target affects the readings.
- Recording the time taken for one full sweep, and discussing the display refresh behaviour that follows from it.

Expected outcome: a working prototype that scans a 150-degree sector and displays detected objects live, with documented accuracy for hard, flat targets, test data, an error table, and a short analysis of the limitations.

14. Troubleshooting

Issue	Check
Serial error / blank display	Correct port selected, baud rate 9600 in both programs, Serial Monitor closed before starting Processing.
No distance reading	TRIG on D10, ECHO on D11, 5V and GND connected, sensor facing the target.
Servo does not move	Signal wire on D12, servo power connected, ground common with the Arduino.
Board restarts / servo jitters	Use a suitable external regulated 5V servo supply with a common ground.
Unstable readings	Use a broad, hard, flat target; avoid angled or soft objects, which reflect or absorb the pulse poorly.
Object shown at the wrong place	Confirm the sensor is mounted squarely on the servo horn, so the physical angle matches the angle in the data.

15. Project Timeline

Week	Activity
1	Literature review, component procurement, software installation
2	Circuit assembly and individual testing of the sensor and servo
3	Arduino firmware development and serial output testing
4	Processing radar display development and integration
5	Calibration, accuracy testing and data collection
6	Analysis, documentation and final report / presentation

16. Estimated Budget

All of the components are common, low-cost parts. Prices vary between suppliers, so the amounts below are to be confirmed at the time of purchase and entered before submission.

Item	Approx. cost (PKR)
Arduino UNO or compatible	To be confirmed
HC-SR04 ultrasonic sensor	To be confirmed
SG90 servo motor	To be confirmed
Breadboard, jumper wires and USB cable	To be confirmed
External regulated 5V supply (optional)	To be confirmed
Total	**To be confirmed**

17. Limitations and Risks

Issue	Mitigation
Servo draws too much current, causing resets or jitter	Use an external regulated 5V supply with a common ground.
Unstable readings from angled or soft surfaces	Test with broad, hard, flat targets and document the effect of the surface.
Wrong serial port, or the port is blocked in Processing	Check the *Serial.list()* output and close the Serial Monitor first.
Narrow sensor beam and 2-degree angle steps may miss thin objects	Use smaller angle steps if needed and report this as a limitation.
Accuracy is affected by temperature and surroundings	Record the test conditions and discuss them in the results.
Only one object is drawn per angle reading	The display shows the nearest reflecting surface at each angle; this is inherent to a single-sensor design.

The prototype is intended for learning and demonstration. It is not a certified safety or navigation device, and it should not be relied on where a missed detection could cause harm.

18. Applications

- Educational demonstrations of ultrasonic ranging, servo control and serial communication.
- Simple obstacle detection and area scanning for small robots.
- Short-range presence and proximity sensing in classroom and laboratory prototypes.

19. Future Scope

- A pan-tilt mount, so the sensor can scan in two axes and build a three-dimensional picture.
- Wireless transmission of the data using Bluetooth or Wi-Fi, removing the USB tether.
- A local LCD or OLED display, plus buzzer or LED proximity alerts, so the system works without a computer.
- Filtering and averaging of readings to reduce noise and false markers.
- A more accurate sensor, such as a time-of-flight or LiDAR module, for longer range and finer resolution.

20. Conclusion

This project provides a practical, low-cost introduction to ultrasonic ranging, servo control, pulse timing, digital input and output, serial communication and real-time visualisation. Building and evaluating the radar system gives hands-on experience of integrating hardware and software, and produces a documented prototype with a defined test procedure and a clear discussion of its limitations.

21. References

1. HC-SR04 ultrasonic sensor datasheet.
2. Arduino documentation: Servo library and the *pulseIn()* function.
3. Processing documentation: Serial library.
4. Arduino Ultrasonic Radar Project Guide (supplied project guide).
5. Project Proposal: Arduino Ultrasonic Radar System (supplied project proposal).

Appendix - Project files

File	Contents
radar.ino	Arduino firmware: servo sweep, distance measurement and serial output (Section 10).
radar_display.pde	Processing sketch: serial input and the radar display (Section 11).
README.md	Project summary, wiring table and run instructions in the GitHub repository.
Project report (PDF)	This formal project report, with the full text also kept as PROJECT_REPORT.md in the repository.